

Certified Short-Cycle Counts for the IEEE 802.11n (WiFi) LDPC Codes

A machine-checked census of the deployed parity-check graphs
via the trace-formula gap law

Carles Marín*

Independent researcher
karlesmarin@gmail.com

June 2026

Abstract

Short cycles in the Tanner graph of a low-density parity-check (LDPC) code degrade belief-propagation decoding, and counting them is a standard task in code design. This paper does not count them faster. It counts them *with a certificate*: for each of the four LDPC codes of the IEEE 802.11n (WiFi) standard at block length $n = 648$, the number of shortest cycles is obtained from a machine-checked identity—the trace-formula gap law, proved ~~sorry~~-free in Lean 4 in the companion paper [11]—and cross-checked by three mutually independent computations that agree exactly: the non-backtracking trace $\text{tr } B^g/2g$, the gap law $(\text{tr } A^g - p_g)/2g$, and direct enumeration. The census reads $c_6 = 3942$ (rate $1/2$), $c_6 = 8046$ (rate $2/3$), $c_4 = 54$ (rate $3/4$, where the girth collapses to 4) and $c_6 = 32\,346$ (rate $5/6$), and it tells a design story: the rate- $3/4$ code pays for its density with four-cycles, the most damaging kind, while the rate- $5/6$ code buys its girth back at the price of thirty-two thousand hexagons. The counting algorithms of the coding-theory literature are faster and reach further; the contribution here is the certification layer—an exact identity backed by a theorem with a checkable proof—which, to the best of my knowledge, no prior cycle-counting tool carries.

Reader’s map. The theorem is borrowed (Section 2; proved in the companion). The new thing is the audited census: three independent routes (Section 3), the four deployed codes (Section 4, Table 1), and the honest accounting of what the certificate does and does not buy (Sections 5–6).

1 Introduction: a number a decoder lives by

What does it mean to *trust* a cycle count? When Gallager invented LDPC codes in 1962 the world had no use for them; thirty years later they returned—to WiFi, to 5G, to deep space—decoded by belief propagation on a bipartite *Tanner graph* [2] whose two sides are the code’s bits and its parity checks [1, 3]. Belief propagation is exact on a tree and only approximate on a graph with cycles: a short cycle lets a message circle back to its origin and reinforce itself, a rumor mistaken for independent confirmation. The shorter the cycle, the tighter the loop, so the *girth* of the Tanner graph—the length of its shortest cycle—and the *number* of cycles at and just above the girth became quantities a code designer measures [2, 4].

*Census computed with AI assistance (Claude, Anthropic); the mathematics, the theorem it rests on, and all claims are the author’s responsibility. The certified identity is proved in the companion paper [11].

Measuring them is a solved problem. Spectral and message-passing algorithms count short cycles efficiently, and on the quasi-cyclic graphs of real standards they count well past the girth [4, 5, 6]. This paper adds nothing to their speed or reach. It adds one thing they lack: a count whose defining identity is a *theorem with a machine-checked proof*. The companion paper [11] proves, **sorry-free** in Lean 4 over Mathlib, that for a finite graph of girth g the number of shortest cycles is

$$c_g = \frac{\text{tr } A^g - p_g}{2g} = \frac{\text{tr } B^g}{2g},$$

with A the adjacency matrix, B the Hashimoto non-backtracking operator, and p_g the g -th power sum of the roots of the matching polynomial. Here we apply that identity, as a *certified census*, to the LDPC codes a WiFi chip actually runs.

2 The identity we apply, and how far it is certified

Which theorem are we leaning on, and where does its guarantee stop? The companion paper [11] proves the *trace-formula gap law*: for every k on the sharp window $1 \leq k \leq g + 1$,

$$\text{tr } A^k - p_k = \text{tr } B^k.$$

Below the girth ($k < g$) both sides vanish; at $k \in \{g, g + 1\}$ both equal $2k c_k$, where c_k is the number of k -cycles. Two consequences are all we need. First, the below-girth vanishing $\text{tr } B^k = 0$ for $k < g$ is a sanity certificate: a Tanner graph of girth g *must* report zero closed non-backtracking walks shorter than g , and any code graph failing it is mis-parsed. Second, at $k = g$ the law is the shortest-cycle count above.

How far is it certified? Both facts are theorems in Lean, depending only on the three standard axioms (`propext`, `Classical.choice`, `Quot.sound`); the below-girth vanishing is “Stone A” and the window equality is the capstone “Stone B” of the companion. So the *identity* that turns spectra and matching counts into a cycle count is machine-checked end to end. What is *not* part of the theorem is the arithmetic on any particular matrix: evaluating $\text{tr } A^g$, p_g or $\text{tr } B^g$ on a concrete 2376-edge Tanner graph is ordinary exact computation, not a Lean proof. That gap—between a certified formula and its evaluation on a specific input—is exactly what the three-route cross-check of the next section is built to close.

3 Three routes to one number

Why compute the same integer three ways? Because a certified *formula* evaluated by fallible code is only as trustworthy as the evaluation, and the cheapest guard against an implementation slip is redundancy by independent method. The census computes each c_g along three routes that share no code path and meet only at the answer.

The three routes (mutually independent)

R1 — non-backtracking trace. Build the Hashimoto operator B on the $2|E|$ darts as a sparse 0/1 matrix and form $\text{tr } B^g$ without ever densifying (powers by sparse products, traces by the Hadamard identity $\text{tr } B^k = \sum (B^a \circ (B^b)^\top)$, $a + b = k$); then $c_g = \text{tr } B^g / 2g$.

R2 — the gap law. Compute $\text{tr } A^g$ from the adjacency matrix and p_g by Newton from matching counts— $p_4 = 2m_1^2 - 4m_2$, $p_6 = 2m_1^3 - 6m_1m_2 + 6m_3$, with $m_1 = |E|$, $m_2 = \binom{m_1}{2} - \sum_v \binom{d_v}{2}$, and m_3 by an exact bipartite edge-deletion recursion; then $c_g = (\text{tr } A^g - p_g) / 2g$.

R3 — direct enumeration. Count cycles combinatorially: for $g = 4$, sum $\binom{\text{codeg}}{2}$ over same-side vertex pairs (bipartite four-cycles); for $g = 6$, a VF2 subgraph search for C_6 , divided by its 12 automorphic rootings.

R1 lives in the geodesic world of the non-backtracking operator; R2 in the spectral-plus- matching world of the gap law; R3 in plain enumeration. They invoke disjoint libraries and disjoint arithmetic. When all three return the same integer, an error would have to be a *coincidence across three methods*—and the gap law is exactly the theorem that says R1 and R2 must agree, so a three-way match also re-confirms the identity on each real graph.

The matching-count route deserves one remark. The recursion $m_3 = \frac{1}{3} \sum_{(u,v) \in E} m_2(G - \{u, v\})$ is evaluated in closed form on the bipartite graph— $N(u)$ and $N(v)$ are disjoint, so the deletion’s matching numbers reduce to degree sums—never copying the graph; this is what keeps R2 a few seconds rather than a few hours on a 2376-edge instance.

4 The census: four codes a WiFi chip runs

What do the deployed codes actually look like? IEEE 802.11n specifies LDPC codes at three block lengths; we take the longest, $n = 648$, with quasi-cyclic lifting size $z = 27$, in all four code rates. Each parity-check matrix yields a Tanner graph on $|V| = n + m$ vertices ($m = n(1 - \text{rate})$ checks) with a constant $|E| = 2376$ edges. The census, with the below-girth zeros confirmed on every code, is Table 1.

rate	$ V $	$ E $	girth g	c_g (R1 = R2 = R3)	shortest cycle
1/2	972	2376	6	3 942	hexagon
2/3	864	2376	6	8 046	hexagon
3/4	810	2376	4	54	square
5/6	756	2376	6	32 346	hexagon

Table 1: The certified short-cycle census of the IEEE 802.11n LDPC codes, $n = 648$, $z = 27$. For each code the three independent routes of Section 3 return the same integer, and the Lean-certified below-girth vanishing $\text{tr } B^k = 0$ ($k < g$) holds on all four Tanner graphs. Computed in exact arithmetic; the pipeline is `research/ldpc-gaplaw/census_pilot2.py`.

The story the four numbers tell. Read down the table and a design trade-off appears (Figure 1). At rate 1/2 the graph is sparse enough to hold girth 6 with the fewest hexagons, 3942—the cleanest of the four. At rate 2/3 the girth survives but the hexagon population doubles. At rate 3/4 the girth *collapses* to 4: the density forces 54 squares, the most damaging cycles a Tanner graph can carry. At rate 5/6, denser still, the designers recover girth 6—trading the squares away—but pay for it with 32 346 hexagons, an eight-fold jump over rate 1/2. The certificate does not judge the trade; it makes the price exact.

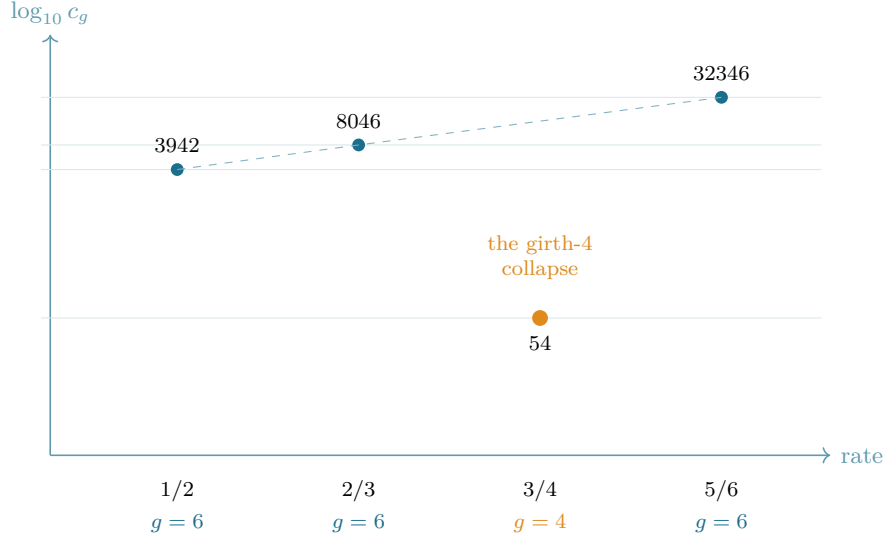


Figure 1: The design landscape of the four IEEE 802.11n codes at $n = 648$. The three girth-6 rates trace a rising hexagon count (dashed); rate 3/4 drops out of that trend because its girth collapses to 4, where the count is small (54) but the cycles are the short, decoder-damaging kind. “Few cycles” and “long cycles” are different goods, and the census separates them.

5 What the certificate buys

Counting short cycles is solved—so what is new here? Not the count, and not its cost. The spectral method of Blake and Lin and the message-passing methods of Karimi and Banihashemi count further (to $2g - 2$ and beyond, with correction terms) and faster, and they exploit the quasi-cyclic block structure that the present pipeline ignores [5, 4, 6]. Against them this census is slower and shorter-reaching.

What it carries that they do not is a *certificate*: the identity producing the number is a theorem with a proof a machine has checked, not a derivation trusted to a paper. Three things follow that matter in practice. The below-girth vanishing is a *free integrity check* on a parsed parity-check matrix—a single mis-read entry that creates a phantom short cycle shows up as a nonzero $\text{tr } B^k$ where the theorem forbids one. The agreement of the non-backtracking and matching routes is not a lucky coincidence but a *predicted* equality, so a mismatch localizes a bug to one route. And the smallest moments of the non-backtracking operator, $\text{tr } B^g = 2g c_g$ and $\text{tr } B^{g+1} = 2(g+1)c_{g+1}$, which govern percolation and epidemic thresholds on a network [7] and the detectability limit in community detection [8], are now pinned exactly from quantities computed by independent, well-tested code—an entry-level regression test for any implementation of B .

6 Honest boundaries

Where exactly does the guarantee stop? In three places, stated plainly.

The window is $k \leq g + 1$. The certified identity counts cycles *at* the girth (and at $g + 1$); the engineering quantities past it—multiplicities up to $2g - 2$, the correction terms of [5, 6]—are not covered by the theorem and are not computed here.

The matrices are third-party. The Tanner graphs are built from parity-check matrices derived from the IEEE 802.11n standard via a public toolkit, not re-derived from the standard’s Annex F.

Their quasi-cyclic block structure ($z = 27$) and degree profiles were cross-checked, and the three-route agreement is itself strong evidence the graphs are the intended ones, but a standards-grade census would re-derive H from the normative document. The certificate covers the *counting identity*, not the provenance of the input.

No algorithmic novelty. The routes are textbook. The contribution is their *composition under a machine-checked identity*, which a literature search on 2026-06-11 found absent: no Lean, Coq, or other proof-assistant work on cycle counting for coding theory. As always, such a claim is made only to the best of one’s knowledge.

7 Open questions

1. *The 5G headline.* The same pipeline applies to the 3GPP NR codes (TS 38.212, base graphs BG1 and BG2 with their deployed lifting sizes). A certified census across the 5G rate-compatible family is the natural next target—larger graphs, the same theorem.
2. *The corrected window in Lean.* Engineers count to $2g - 2$ with closed-form corrections; the companion paper leaves the first correction term ($k = g + 2$, the lollipop walks) as an open formalization. A certified count that reaches $2g - 2$ would match the literature’s reach *with* a proof.
3. *Re-derivation from the standard.* Building H directly from IEEE 802.11n Annex F and 3GPP TS 38.212, inside the certified pipeline, would close the provenance gap and make the census standards-grade end to end.
4. *The certificate as a design instrument.* The pair $(\text{tr } B^g, \text{tr } B^{g+1})$ is a cheap certified invariant of a Tanner graph. How much of a code’s short-cycle profile—and of its decoding behavior—does this certified pair pin down, and where does it stop distinguishing codes that decode differently?

Data and code availability

The parity-check alist files, the census pipeline (`research/ldpc-gaplaw/census_pilot2.py`) and the full output are in the public repository <https://github.com/karlesmarin/godsil-gutman-1ean>, alongside the Lean sources of the companion paper [11] that prove the identity applied here.

References

- [1] R. G. Gallager, *Low-density parity-check codes*, IRE Trans. Inform. Theory **8** (1962) 21–28.
- [2] R. M. Tanner, *A recursive approach to low complexity codes*, IEEE Trans. Inform. Theory **27** (1981) 533–547.
- [3] D. J. C. MacKay, *Good error-correcting codes based on very sparse matrices*, IEEE Trans. Inform. Theory **45** (1999) 399–431.
- [4] M. Karimi, A. H. Banihashemi, *Message-passing algorithms for counting short cycles in a graph*, IEEE Trans. Commun. **61** (2013) 485–495.
- [5] I. F. Blake, S. Lin, *On short cycle enumeration in biregular bipartite graphs*, IEEE Trans. Inform. Theory **64** (2018) 6526–6535.

- [6] A. Dehghan, A. H. Banihashemi, *On computing the multiplicity of cycles in bipartite graphs using the degree distribution and the spectrum of the graph*, IEEE Trans. Inform. Theory **65** (2019) 3778–3789.
- [7] B. Karrer, M. E. J. Newman, L. Zdeborová, *Percolation on sparse networks*, Phys. Rev. Lett. **113** (2014) 208702.
- [8] F. Krzakala, C. Moore, E. Mossel, J. Neeman, A. Sly, L. Zdeborová, P. Zhang, *Spectral redemption in clustering sparse networks*, Proc. Natl. Acad. Sci. **110** (2013) 20935–20940.
- [9] K. Hashimoto, *Zeta functions of finite graphs and representations of p -adic groups*, Adv. Stud. Pure Math. **15** (1989) 211–280.
- [10] The mathlib community, *The Lean mathematical library*, in: CPP 2020, ACM, 2020, 367–381.
- [11] C. Marín, *The walks that remember the cycles: a machine-checked sharp gap law between the matching polynomial and the non-backtracking spectrum in Lean 4* (Part VI of the godsil–gutman–lean series), Zenodo, 2026.